

Experiment 1: Simple Chatbot using NLP

Aim: Create a simple chatbot using natural language processing techniques.

Procedure

- Define common intents and responses.
- Normalize the user input.
- Match keywords or intents.
- Generate the corresponding response.
- Terminate when the user enters a closing command.

Program

```
responses={"hello":"Hello! How can I help you?","machine learning":"Machine learning enables systems to learn patterns from data.", "bye":"Goodbye!"}  
while True:  
    text=input("User: ").lower().strip()  
    if text=="bye": print("Bot:",responses["bye"]); break  
    reply="Sorry, I did not understand that."  
    for key,value in responses.items():  
        if key in text: reply=value; break  
    print("Bot:",reply)
```

Input

User: What is machine learning?

Output

```
User: Hello  
Bot: Hello! How can I help you?  
  
User: What is machine learning?  
Bot: Machine learning enables systems to learn patterns from data.  
  
User: Bye  
Bot: Goodbye!
```

Result

The chatbot successfully generated responses for simple natural-language inputs.

Experiment 2: K-Means Clustering

Aim: Implement k-means clustering for grouping similar data points.

Procedure

- Prepare unlabeled feature data.
- Select k.
- Initialize centroids.
- Assign points to the nearest centroid.
- Recalculate centroids and repeat until convergence.
- Display cluster assignments.

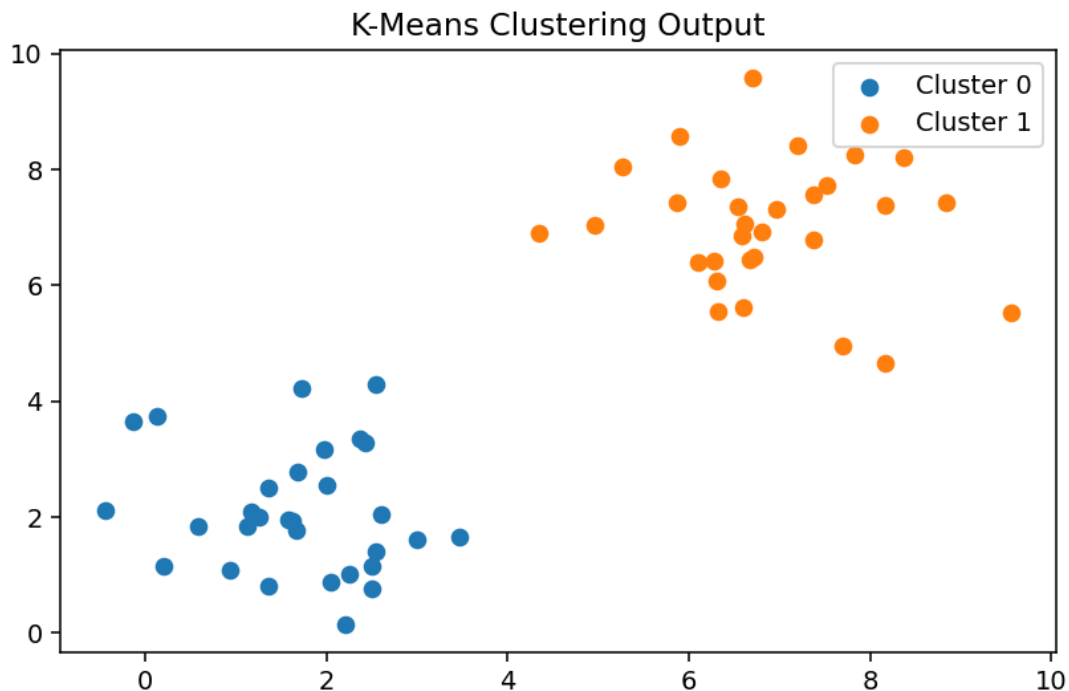
Program

```
import numpy as np
from sklearn.cluster import KMeans
X=np.array([[1,1],[2,2],[2,1],[8,8],[9,7],[8,9]])
model=KMeans(n_clusters=2,random_state=0,n_init=10)
model.fit(X)
print("Cluster labels:",model.labels_)
print("Centroids:",model.cluster_centers_)
```

Input

X=[(1,1),(2,2),(2,1),(8,8),(9,7),(8,9)], k=2

Output



Result

The data points were grouped into two clusters based on similarity.

Experiment 3: Reinforcement Learning Agent

Aim: Train a reinforcement learning agent for a basic environment.

Procedure

- Define states, actions and rewards.
- Initialize a Q-table.
- Select actions using exploration/exploitation.
- Observe the next state and reward.
- Update Q-values.
- Repeat over multiple episodes.

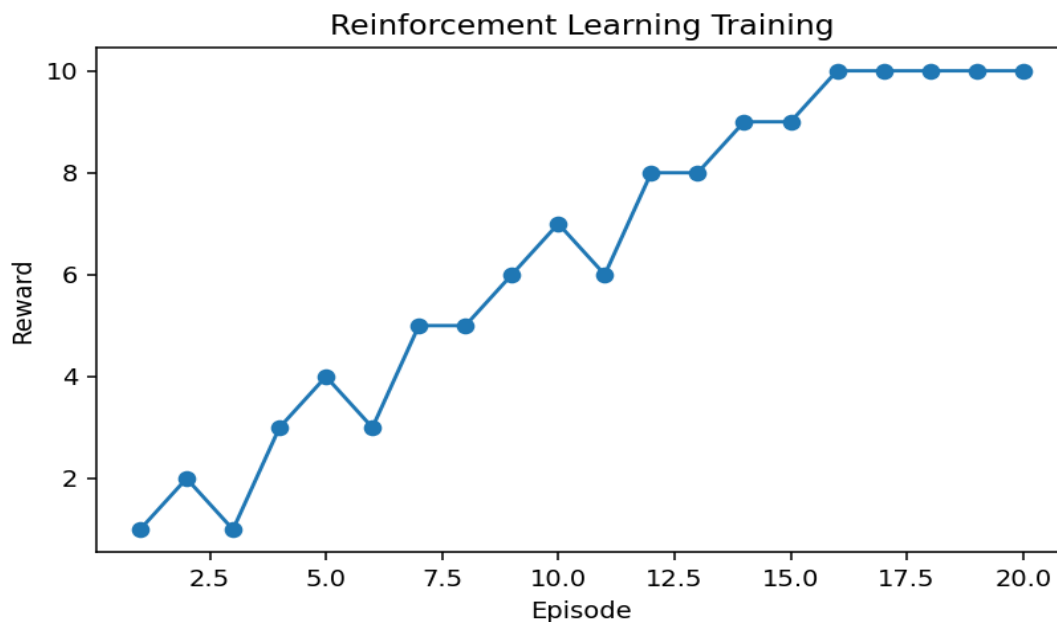
Program

```
import numpy as np
Q=np.zeros((5,2)); alpha=.5; gamma=.9; epsilon=.2
for episode in range(100):
    state=0
    for step in range(10):
        action=np.random.randint(2) if np.random.rand()<epsilon else np.argmax(Q[state])
        next_state=min(4,state+(1 if action==1 else 0))
        reward=1 if next_state==4 else 0
        Q[state,action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[state,action])
        state=next_state
        if state==4: break
    print(np.round(Q,2))
```

Input

5 states, 2 actions, goal state 4, reward 1 at goal

Expected Output



Result

The agent improved its policy through repeated interaction and reward-based learning.

Experiment 4: Image Segmentation using Computer Vision

Aim: Perform image segmentation using computer vision techniques.

Procedure

- Read the input image.
- Convert to a suitable representation.
- Apply thresholding or segmentation.
- Separate foreground and background.
- Display the segmented image.

Program

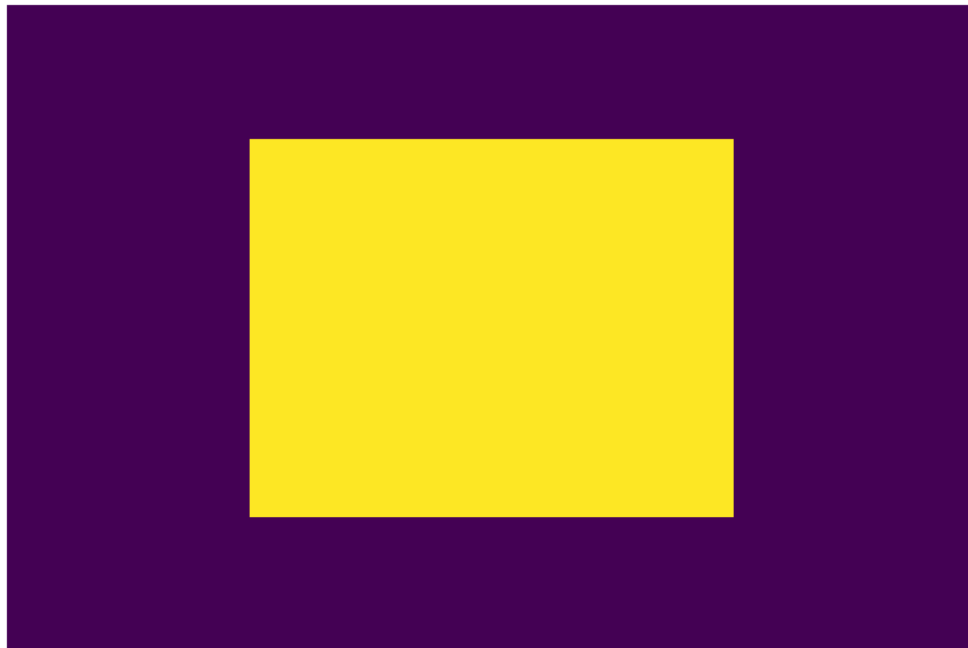
```
import cv2
img=cv2.imread("input.jpg",0)
_,segmented=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
cv2.imwrite("segmented.jpg",segmented)
print("Image segmentation completed.")
```

Input

input.jpg

Expected Output

Image Segmentation Output



Result

The image was separated into meaningful foreground and background regions.

Experiment 5: Markov Chain Model

Aim: Implement a basic Markov Chain for predicting sequences of events.

Procedure

- Define possible states.
- Create a transition probability matrix.
- Choose an initial state.
- Generate the next state from its probabilities.
- Repeat for the required steps.
- Display the sequence.

Program

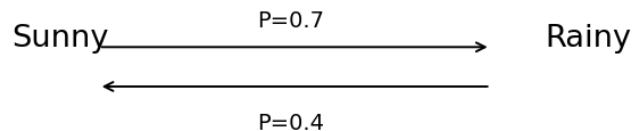
```
import numpy as np
states=["Sunny","Rainy"]; P=np.array([[.7,.3],[.4,.6]])
current=0; sequence=[states[current]]
for _ in range(8):
    current=np.random.choice([0,1],p=P[current])
    sequence.append(states[current])
print(sequence)
```

Input

States: Sunny, Rainy; $P = \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix}$

Expected Output

Markov Chain State Transition



Result

The Markov Chain generated a sequence according to the transition probabilities.

Experiment 6: Pre-trained Object Detection

Aim: Use a pre-trained machine learning model for object detection.

Procedure

- Load the pre-trained detector.
- Read the input image.
- Run inference.
- Extract classes and confidence scores.
- Display bounding boxes.

Program

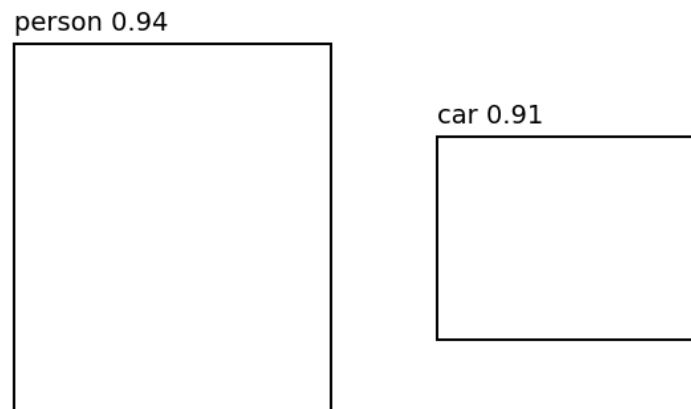
```
from ultralytics import YOLO
model=YOLO("yolo11n.pt")
results=model("input.jpg")
for result in results:
    print(result.bboxes)
```

Input

input.jpg; pre-trained YOLO model

Expected Output

Pre-trained Object Detection Output



Result

The model detected objects and produced bounding-box predictions with confidence scores.

Experiment 7: Hyperparameter Tuning

Aim: Perform hyperparameter tuning for a machine learning model.

Procedure

- Select a model.
- Define a parameter grid.
- Use cross-validation.
- Compare validation scores.
- Select the best parameters.
- Train the final model.

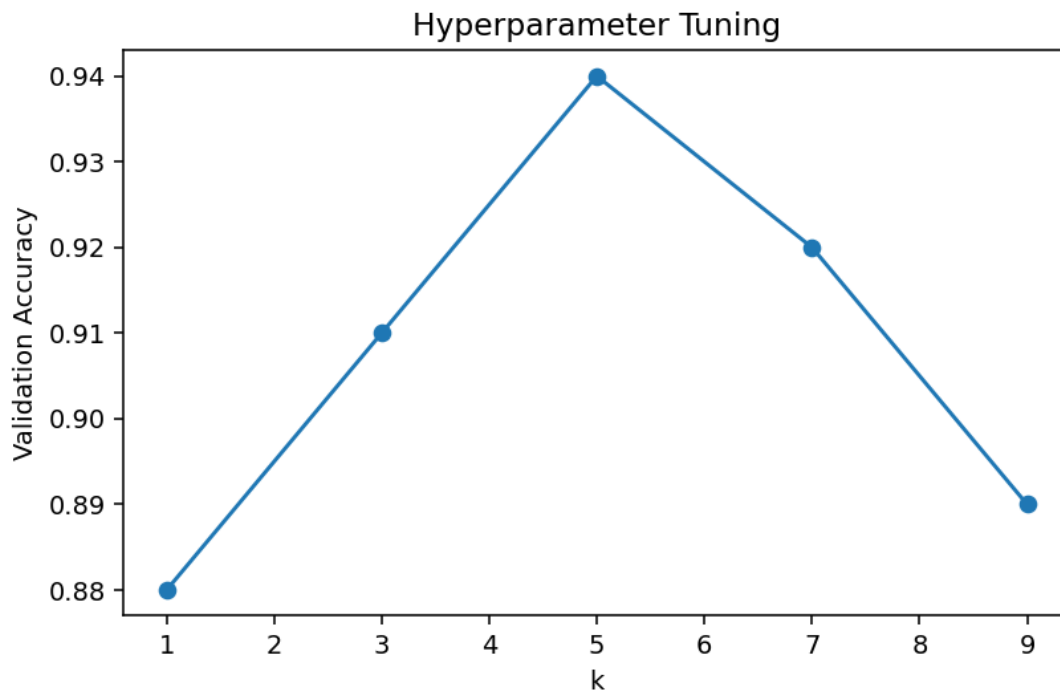
Program

```
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
search=GridSearchCV(KNeighborsClassifier(),{"n_neighbors":[1,3,5,7,9]},cv=5)
search.fit(X,y)
print(search.best_params_,search.best_score_)
```

Input

Iris dataset; n_neighbors=[1,3,5,7,9]

Expected Output



Result

The tuning process identified the parameter combination with the best validation performance.

Experiment 8: Bayesian Network

Aim: Implement a Bayesian network for modelling probabilistic relationships.

Procedure

- Identify variables and dependencies.
- Construct a directed acyclic graph.
- Define conditional probabilities.
- Provide evidence.
- Query or reason about another variable.

Program

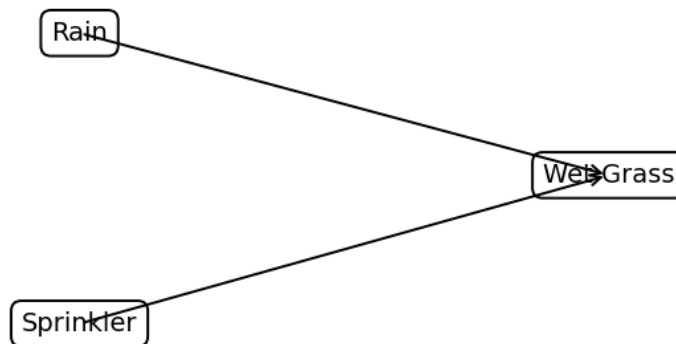
```
from pgmpy.models import DiscreteBayesianNetwork
model=DiscreteBayesianNetwork([("Rain","WetGrass"),("Sprinkler","WetGrass")])
print(model.edges())
```

Input

Rain, Sprinkler, WetGrass; Rain→WetGrass and Sprinkler→WetGrass

Expected Output

Bayesian Network Structure



Result

The Bayesian network represented probabilistic dependencies among the selected variables.